

助教分配30组-作业报告

星域防御矩阵

项目	内容
课程作业	基于 Qt Widgets 与 QGraphicsScene 的塔防类小游戏
小组成员	马若颜，邓宇轩，刘可凡
小组分工	马若颜：代码实现；邓宇轩：界面设计与问题修复；刘可凡：调试

一、程序功能介绍

本项目实现了一款科技风格的塔防游戏。玩家在主菜单中进入游戏后，需要在五行九列的防御网格上布置不同功能的防御单位，抵御从右侧不断出现的敌人。游戏通过金币资源、卡牌选择、冷却时间、敌人波次和技能按钮构成主要玩法循环。

游戏主界面由 `QGraphicsView` 显示场景内容，使用 `QTimer` 以固定时间间隔驱动全局更新。玩家点击“Start Wave”开始敌人波次；每清空一波敌人后，系统会自动进入下一波，并根据波次增加敌人生成预算，使游戏难度逐步提升。界面上方显示当前波次、总分和剩余生成队列。

资源系统以金币为核心。玩家初始拥有一定金币，商店会随时间自动增加金币，金币生产单位也可以定期产出金币。玩家通过卡牌选择要放置的单位，放置时系统会检查金币是否足够、目标格子是否空闲，并扣除对应费用。

防御单位包含金币生产、远程攻击、防护阻挡和爆炸伤害等不同类型。射击单位会在同一行检测前方敌人并发射子弹；防护单位拥有较高生命值，主要用于阻挡敌人；炸弹单位可造成范围伤害；金币生产单位负责补充资源。单位还支持升级逻辑，升级后可提升攻击、生命、收益或范围等属性。

辅助技能包括 `FREEZE`、`POWER` 和 `SHOVEL`。`FREEZE` 可对场上敌人施加减速；`POWER` 可强化指定防御单位，例如提高射击速度、恢复防护单位生命或触发炸弹复制效果；`SHOVEL` 用于移除已放置的防御单位，方便玩家调整阵型。敌人突破左侧防线时游戏结束，并显示最终得分。

二、项目各模块与类设计细节

项目采用 `Qt Widgets + Graphics View` 的结构组织。窗口层负责菜单和游戏场景切换，游戏对象基本继承自 `QGraphicsItem` 或 `QGraphicsObject`，通过 `paint`、`advance`、`boundingRect`、`collidesWithItem` 和鼠标事件实现绘制、更新、碰撞与交互。

模块	主要类	设计职责
窗口与流程控制	<code>MainWindow</code> 、 <code>GameWindow</code>	<code>MainWindow</code> 负责开始菜单、规则弹窗和进入游戏； <code>GameWindow</code> 创建 <code>QGraphicsScene</code> 、网格、商店、按钮和卡牌，并通过 <code>QTimer</code> 驱动场景更新、波次生成、暂停和返回。
地图网格模块	<code>Map</code>	表示可放置单位的格子，处理鼠标点击、放置合法性检查、金币扣除、格子占用判断和防御单位实例化。
商店与卡牌模块	<code>Shop</code> 、 <code>Card</code>	<code>Shop</code> 保存金币数量并定时增长； <code>Card</code> 维护卡牌费用、冷却、位置、编号和当前选中状态，是玩家选择防御单位的入口。
技能按钮模块	<code>Button</code>	实现 <code>FREEZE</code> 、 <code>POWER</code> 、 <code>SHOVEL</code> 三种按钮状态、冷却遮罩和点击效果，通过静态变量记录当前是否处于强化或铲除模式。

模块	主要类	设计职责
防御单位基类	Paotai	抽象出生命值、攻击力、等级、放置费用、升级按钮、升级消耗、动画贴图和基础碰撞形状，为各类防御单位复用公共逻辑。
防御单位子类	CoinMaker、Shooter、Protector、Bomb	CoinMaker 负责金币收益；Shooter 负责检测同一行敌人并发射子弹；Protector 负责承受伤害和恢复；Bomb 负责范围爆炸、升级扩大范围和强化复制。
敌人模块	Enemy、NormalEnemy、PlusEnemy	Enemy 保存生命、攻击、速度、状态、减速标记和总分等公共数据；普通敌人与强化敌人分别实现不同的移动、攻击和特殊行为。
飞行物与通用对象	Bullet、Coin、other	Bullet 负责移动、碰撞和伤害结算；other 提供非炮台图形对象的公共类型入口；资源对象统一由 qrc 管理。

2.1 场景与游戏循环

GameWindow 在构造函数中创建 1400 x 650 的场景，添加商店、卡牌、技能按钮和 5 x 9 地图格，并设置 16 毫秒定时器。每次定时器触发时调用 updateGame，内部执行 scene->advance()，让所有图元按统一节奏更新。该设计减少了对象之间的直接耦合，使敌人移动、子弹飞行、植物生产和按钮冷却都能放在各自类的 advance 函数中完成。

波次系统使用 currentWave、spawnQueue、spawnInterval 和 spawnFrameCounter 管理。每波开始时根据波次计算预算，再随机生成普通敌人或强化敌人队列。队列清空且场上没有存活敌人时自动进入下一波。

2.2 放置、升级与资源结算

玩家先点击 Card 选择单位，再点击 Map 网格完成放置。Map 会根据场景坐标对齐到网格中心，检查目标位置是否已有 Paotai 对象，并从 Shop 中扣除金币。Paotai 基类统一处理升级按钮和金币消耗；具体子类重写 upgrade，根据自身定位提升不同属性，例如 Shooter 提升攻击与射速，Protector 提升生命，CoinMaker 提升收益，Bomb 提升伤害与爆炸范围。

2.3 碰撞与战斗逻辑

战斗主要依靠 QGraphicsItem 的碰撞机制和自定义判定。Shooter 在前方同一行发现敌人后创建 Bullet；Bullet 每帧向右移动，碰到 Enemy 后扣除生命并删除自身。Bomb 通过范围判定对周围敌人造成伤害。敌人根据状态在行进、攻击、死亡或燃烧之间切换，死亡后累计得分；如果敌人越过防线，则触发游戏结束。

三、小组成员分工情况

成员	主要分工	具体工作内容
马若颜	代码实现	负责项目部分代码编写，卡牌放置、敌人波次、防御单位、子弹碰撞、分数与游戏结束等核心逻辑。

成员	主要分工	具体工作内容
刘可凡	调试与问题修复	负责运行测试、定位编译和运行问题，修复了游戏崩溃，黑屏以及卡片冷却的问题
邓宇轩	界面设计与bug修复	负责主菜单和游戏界面设计，整理图片与动效资源，设计视觉风格，调整按钮、卡牌、商店、地图网格、状态文字和动画扫描线等界面表现。修改了初版代码，修复了子弹，铲子，炸弹的bug

三位成员在开发过程中采用分工协作方式推进：代码实现先搭建可运行的游戏骨架，界面设计同步准备视觉资源和交互布局，调试同学在每个功能合入后进行构建与运行验证。遇到问题时，小组通过复现错误、缩小范围、修改配置或逻辑、再次验证的流程解决。

四、项目总结与反思

本项目基本完成了塔防游戏的核心玩法闭环：菜单进入游戏、选择卡牌、消耗金币布置单位、敌人按波次出现、防御单位攻击或阻挡敌人、技能辅助、升级强化、计分和失败判定。通过 Qt 的 Graphics View 框架，各类对象都可以在统一场景中绘制和更新，比较适合实现这种由大量图元组成的小游戏。

项目的主要收获是加深了对 Qt 事件机制、图形项绘制、定时器更新、资源文件管理和面向对象设计的理解。Paotai 与 Enemy 两个基类让不同单位共享公共属性和行为，再通过子类重写实现差异化能力，体现了继承和多态在游戏开发中的实际作用。

开发过程中也暴露出一些不足。第一，部分注释和文本出现编码异常，说明项目文件编码需要统一管理；第二，一些全局状态通过静态变量传递，虽然实现简单，但随着功能增多会增加维护难度；第三，数值平衡仍有优化空间，例如敌人波次预算、防御单位费用、技能冷却和升级收益可以通过更多测试进一步调整。

后续如果继续完善，可以考虑增加开始前的难度选择、更多敌人类型、存档与排行榜、关卡编辑器、音效开关和更清晰的胜利目标。同时，可以把配置数据从代码中抽离到表格或 JSON 文件，减少硬编码，使后续调参和扩展更加方便。